Django Assignment: GitHub Profile Viewer

Intermediate level | Time: 1.5 hours | Total: 100 marks

1. Objective

Build a Django app that searches GitHub users through the public GitHub REST API, displays their profile and repositories, and lets logged-in users bookmark profiles. You will practice calling a third-party API with **requests**, handling errors, saving API data in models, and building views, forms and templates.

2. Learning Outcomes

- Call an external REST API with timeouts and error handling
- Write a reusable service layer, separate from views
- Store API data in a model with constraints (no duplicate bookmarks)
- Use authentication, `messages`, and POST-only actions
- Render API data in templates

3. Suggested Time Plan

Time	Task
0:00 - 0:10	Project setup, install requests, create app, copy services.py
0:10 - 0:30	Test the service in Django shell, build the search view and template
0:30 - 0:50	BookmarkedProfile model, migrations, admin, bookmark view
0:50 - 1:10	Bookmark list page, delete, login/logout, messages in template
1:10 - 1:30	Test error cases, polish, bonus (caching) if time remains

4. Setup

```
python -m venv venv
source venv/bin/activate          # Windows: venv\Scripts\activate
pip install django requests
django-admin startproject config .
python manage.py startapp profiles
```

Add `'profiles'` to `INSTALLED_APPS`. Add `LOGIN_URL = 'login'` and include `django.contrib.auth.urls` in your project urls (path `accounts/`). No API key is needed. GitHub allows 60 unauthenticated requests per hour per IP.

5. Requirements

Functional

- **Search page** (`/`): form with a username field. On success show avatar, name, bio, followers, public repo count, and the 10 most recently updated repos (name, language, stars, link).
- **Error handling**: show a friendly message for user not found, rate limit exceeded, timeout, and no connection. The site must never show a 500 page for these.
- **Bookmark**: logged-in users can bookmark the viewed profile (POST only). The same user cannot bookmark the same GitHub username twice.
- **Bookmarks page**: list the user's bookmarks (newest first) with a remove button. Users only see their own.
- **Admin**: register BookmarkedProfile with list_display and search.

Technical

- All API calls live in `profiles/services.py`, never inside views or templates
- Every request uses a **timeout**
- Use `unique_together` (or UniqueConstraint) on (user, username)
- Use `@login_required` and `@require_POST` where appropriate

6. Core Code (provided)

Copy these files into your project. Read them carefully: you must be able to explain every part in the viva.

6.1 profiles/services.py (all API fetching logic)

```
import re
import requests

API_BASE = "https://api.github.com"
TIMEOUT = 5  # seconds. Never call an API without a timeout.
HEADERS = {
    "Accept": "application/vnd.github+json",
    "User-Agent": "django-github-viewer",   # GitHub requires a User-Agent
}
# GitHub usernames: letters, digits, single hyphens, max 39 chars
USERNAME_RE = re.compile(r"^[A-Za-z0-9](?:[A-Za-z0-9]|-(?=[A-Za-z0-9])){0,38}$")


# ---- Custom exceptions: views catch these instead of raw requests errors ----
class GitHubError(Exception):
    """Base class for all GitHub related problems."""


class UserNotFound(GitHubError):
    pass


class RateLimited(GitHubError):
    pass


class GitHubUnavailable(GitHubError):
    pass


def _get(path, params=None):
    """Call the GitHub API and return parsed JSON, or raise a GitHubError."""
    url = f"{API_BASE}{path}"
    try:
        response = requests.get(url, headers=HEADERS, params=params, timeout=TIMEOUT)
    except requests.exceptions.Timeout:
        raise GitHubUnavailable("GitHub took too long to respond. Try again.")
    except requests.exceptions.RequestException:
        raise GitHubUnavailable("Could not connect to GitHub. Check your internet.")

    if response.status_code == 404:
        raise UserNotFound("No GitHub user with that username.")

    if response.status_code in (403, 429):
        if response.headers.get("X-RateLimit-Remaining") == "0":
            raise RateLimited("GitHub rate limit reached. Try again in a few minutes.")
        raise GitHubUnavailable("GitHub refused the request.")

    if response.status_code != 200:
        raise GitHubUnavailable(f"GitHub returned an error ({response.status_code}).")

    try:
        return response.json()
    except ValueError:
        raise GitHubUnavailable("GitHub sent an invalid response.")


def get_user(username):
    """Return a clean dict of profile data. Keys match BookmarkedProfile fields."""
    username = (username or "").strip()
    if not USERNAME_RE.match(username):
        raise UserNotFound("That is not a valid GitHub username.")

    data = _get(f"/users/{username}")
    return {
```

```
        "username": data["login"],              # correct capitalisation from GitHub
        "name": data.get("name") or "",         # may be null
        "avatar_url": data.get("avatar_url", ""),
        "bio": data.get("bio") or "",           # may be null
        "public_repos": data.get("public_repos", 0),
        "followers": data.get("followers", 0),
        "html_url": data.get("html_url", ""),
    }


def get_repos(username, limit=10):
    """Return the most recently updated public repos as a list of dicts."""
    data = _get(
        f"/users/{username}/repos",
        params={"sort": "updated", "direction": "desc", "per_page": limit},
    )
    return [
        {
            "name": repo["name"],
            "description": repo.get("description") or "",
            "language": repo.get("language") or "n/a",
            "stars": repo.get("stargazers_count", 0),
            "html_url": repo.get("html_url", ""),
        }
        for repo in data
    ]
```

6.2 profiles/models.py

```
from django.contrib.auth.models import User
from django.db import models


class BookmarkedProfile(models.Model):
    user = models.ForeignKey(User, on_delete=models.CASCADE, related_name="bookmarks")
    username = models.CharField(max_length=39)          # GitHub username
    name = models.CharField(max_length=150, blank=True)
    avatar_url = models.URLField(blank=True)
    bio = models.TextField(blank=True)
    public_repos = models.PositiveIntegerField(default=0)
    followers = models.PositiveIntegerField(default=0)
    html_url = models.URLField(blank=True)
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        unique_together = ("user", "username")
        ordering = ["-created_at"]

    def __str__(self):
        return f"{self.username} (saved by {self.user})"
```

6.3 profiles/views.py

```
from django.contrib import messages
from django.contrib.auth.decorators import login_required
from django.shortcuts import get_object_or_404, redirect, render
from django.views.decorators.http import require_POST

from . import services
from .models import BookmarkedProfile


def search(request):
    query = request.GET.get("q", "").strip()
    context = {"query": query, "profile": None, "repos": []}

    if query:
        try:
            context["profile"] = services.get_user(query)
            context["repos"] = services.get_repos(context["profile"]["username"])
        except services.GitHubError as exc:      # catches all 4 error types
            messages.error(request, str(exc))
```

```
    return render(request, "profiles/search.html", context)


@login_required
@require_POST
def bookmark(request, username):
    try:
        data = services.get_user(username)
    except services.GitHubError as exc:
        messages.error(request, str(exc))
        return redirect("search")

    lookup = data.pop("username")
    _, created = BookmarkedProfile.objects.get_or_create(
        user=request.user, username=lookup, defaults=data
    )
    if created:
        messages.success(request, f"Bookmarked {lookup}.")
    else:
        messages.info(request, f"{lookup} is already bookmarked.")
    return redirect("bookmarks")


@login_required
def bookmark_list(request):
    items = BookmarkedProfile.objects.filter(user=request.user)
    return render(request, "profiles/bookmarks.html", {"bookmarks": items})


@login_required
@require_POST
def bookmark_delete(request, pk):
    item = get_object_or_404(BookmarkedProfile, pk=pk, user=request.user)
    item.delete()
    messages.success(request, "Bookmark removed.")
    return redirect("bookmarks")
```

6.4 profiles/urls.py (include in config/urls.py)

```
from django.urls import path
from . import views

urlpatterns = [
    path("", views.search, name="search"),
    path("bookmark/<str:username>/", views.bookmark, name="bookmark"),
    path("bookmarks/", views.bookmark_list, name="bookmarks"),
    path("bookmarks/<int:pk>/delete/", views.bookmark_delete, name="bookmark_delete"),
]
```

6.5 profiles/admin.py

```
from django.contrib import admin
from .models import BookmarkedProfile

@admin.register(BookmarkedProfile)
class BookmarkedProfileAdmin(admin.ModelAdmin):
    list_display = ("username", "user", "public_repos", "followers", "created_at")
    search_fields = ("username", "name", "user__username")
```

7. Your Tasks (not provided)

- Create `base.html` showing Django `messages` (success and error) and nav links.
- Create `profiles/search.html`: search form (GET, field q), profile card, repo list, and a POST bookmark form with `{% csrf_token %}` (visible only if logged in).
- Create `profiles/bookmarks.html`: list with avatar, username link, and a POST delete button.
- Run migrations, create a superuser, and create login/logout templates (`registration/login.html`).
- Test in the shell before building templates (see below).

Quick shell test

```
python manage.py shell
>>> from profiles import services
>>> services.get_user("torvalds")
>>> services.get_repos("torvalds", limit=3)
>>> services.get_user("this-user-does-not-exist-98765")   # should raise UserNotFound
```

8. Test Checklist

Test	Expected result
Search `torvalds`	Profile and 10 repos displayed
Search `this-user-does-not-exist-98765`	Friendly 'not found' message, no crash
Search `bad name!!`	'Not a valid username' message, no API call made
Turn off Wi-Fi and search	'Could not connect' message
Bookmark the same user twice	Second time shows 'already bookmarked', one DB row
Logged out, open /bookmarks/	Redirected to login
User A tries deleting User B's bookmark id	404 page

9. Bonus (+10 marks)

- **Cache API responses for 5 minutes** using Django's cache framework (see snippet).
- Show a 'Saved' badge on the search page if the profile is already bookmarked.
- Add a `note` field to bookmarks and let the user edit it with a ModelForm.

```
# In services.get_user(), wrap the API call:
from django.core.cache import cache

cache_key = f"gh_user_{username.lower()}"
data = cache.get(cache_key)
if data is None:
    data = _get(f"/users/{username}")
    cache.set(cache_key, data, 300)       # 300 seconds
```

10. Grading Rubric (100 marks)

Criteria	Marks	Full marks when
API integration (services.py)	20	Understood and working; timeout and all error types handled
Search and profile display	15	Profile and repos render correctly, empty values handled
Model and migrations	10	Correct fields, unique constraint, admin works
Bookmark create/list/delete	20	Works, POST only, no duplicates, users see only their own
Authentication and security	10	login_required used, CSRF tokens present, ownership checked
Templates and UX	10	Clean layout, messages shown, links work
Code quality	5	Readable, API logic only in services.py, sensible names
Viva (explain your code)	10	Can explain the service layer, exceptions, and one view
Bonus	+10	Caching, saved badge, or editable notes

11. Submission

- Zip the project (exclude `venv/` and `db.sqlite3`) or push to GitHub.
- Include `requirements.txt` (`pip freeze > requirements.txt`).
- Add a short README with run instructions.

Common Mistakes

- Forgetting `timeout=` in requests, so the page hangs
- Forgetting the `User-Agent` header, so GitHub returns 403
- Using a GET link for bookmark/delete instead of a POST form
- Showing raw exceptions to users instead of messages
- Hammering the API while testing and hitting the 60/hour limit (test in the shell, or use the cache bonus)